

Orchestration

The agentic layer — where a model becomes an agent

plan → execute → review → memory → loop

Agenda

1. **Why orchestration?** — a model is one piece · the loop
2. **Planning** — reasoning models, test-time compute, stop-and-ask
3. **Execution** — tool calling and MCP, local & remote
4. **Review** — reflection, grounded verification, when to stop
5. **Memory** — the four kinds, scope, file demo → mem0
6. **Putting it together** + Q&A

0 · Why orchestration?

A model is one piece, not the whole system.

A model alone is a chat box

One prompt in → one answer out. That's a **chat interface**.

Add four things and it becomes an **agent**:

- **Planning** — break an intent into steps
- **Execution** — call tools to act on the real world
- **Review** — check the work, self-correct
- **Memory** — carry state across steps and sessions

Orchestration is the *mechanism*; "agentic" is the *behavior* you build on it.

This layer is the **brain / control plane** of the stack.

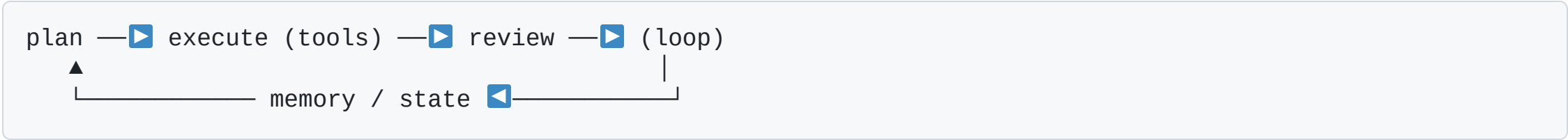
Two kinds of orchestration — use both

Kind	What it is	Strength
Static / deterministic	Fixed workflows, pipelines, routing	Reliable, predictable, auditable
Agentic / autonomous	Agents decide, escalate, adapt	Judgment, exceptions, multi-step

A static foundation gives reliability; the agentic layer fills the thinking and decision-making that fixed flows can't handle.

You don't pick one — most robust systems are mostly **static**, with **agentic** judgment at the hard joints.

The loop



Box	Role	We'll cover
Plan	Thinking	reasoning, decomposition, stop-and-ask
Execute	Acting	tool calling, MCP
Review	Checking	reflection, grounded verification
Memory	State	working / episodic / semantic / procedural

The running example — an SRE agent

14:35 — an alert fires. `checkout-api` is throwing a **spike of 5xx**.

The SRE just states an **intent**: *"checkout-api is 5xx, deal with it."*

Combine both: keep your alerting pipeline; **add an agent on top** that —

- **reads the same alert** and **reasons** to a root cause (*plan*)
- **stops to ask** before a risky rollback (*plan*)
- **acts** — calls a real tool (*execute*)
- **verifies** the fix against live metrics (*review*)
- **remembers** the lesson → less manual toil next time (*memory*)

1 • Planning

The agent thinks, pauses, and asks.

Who writes the steps?

TRADITIONAL SOFTWARE

Runbook written AHEAD,
reviewed/compiled first.

```
if error_rate > 5%:  
    restart_pods()  
elif latency > 1s:  
    scale_up()
```

Runs only the script you
foresaw. Novel case → stuck.

AGENT (SRE on-call)

User states an INTENT:
"checkout-api 5xx, fix it"



MODEL invents steps AT RUN-TIME:

1. when did 5xx start?
2. any deploy near then?
3. read logs → find cause
4. (rollback? → ASK human)
5. roll back, then re-verify

Traditional: a programmer types every branch at *compose-time*.

Agent: the user gives an *intent*; the model invents the steps at *run-time*.

Reasoning is built from tokens — one at a time

Planning = the model **reasoning** an intent into steps. And reasoning bottoms out in how a model generates *anything*:

Autoregressive: at each step, look at *all* text so far → pick the next token → append → repeat.

```
"The capital of France is" → [model] → next: "Paris" (highest prob)
append → "The capital of France is Paris" → generate the next token...
```

Everything a model "does" is this single loop. So what is *reasoning*?

Ordinary vs reasoning model

ORDINARY MODEL:

prompt —▶ [blurts an answer] —▶ "Try restarting the pods."
(fast guess, easily wrong about the real cause)

REASONING MODEL:

prompt —▶ [emits a PILE of "thinking" tokens first] —▶ then concludes

scratchpad

"5xx rising since 14:32. What changed? → deploy v2.3
at 14:30. Logs: 'DB connection timeout'. v2.3 cut
the pool to 5. Cause = the deploy, NOT traffic."

↓

"Cause: deploy v2.3 shrank the DB pool. Fix: roll back."

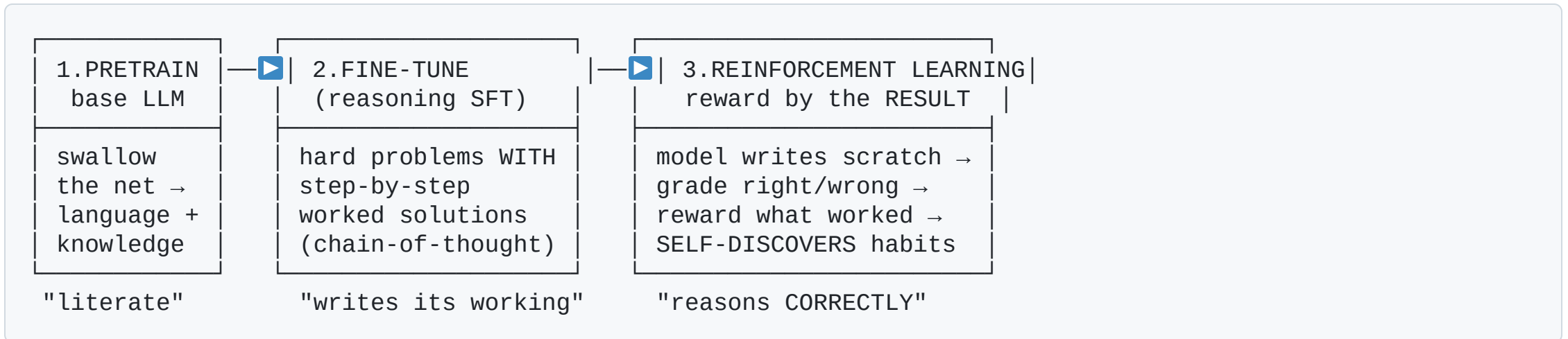
That chain of thinking tokens (**chain-of-thought**) is the reasoning.

Four things that matter (at the token level)

1. **Thinking tokens are ordinary tokens** — same next-token loop. No magic; a draft *before* the answer.
2. **Each token feeds back and steers the next.** The model writes **notes to itself**. "Oh, that's wrong, back up" really redirects what follows.
3. **Each token = one forward pass = compute.** More scratchpad = **more compute** spent → **test-time compute**. "Think longer" literally = *generate more tokens before committing*.
4. **Trained by RL, not imitation.** Rewarded for the *right answer*, so it **discovers** habits: try paths, double-check, back out of dead ends.

How a reasoning model is trained (3 stages)

You don't train from scratch — you take an ordinary LLM and train it *further*.



Stage 2 teaches the **form** (show your steps); stage 3 teaches the **substance** (reason to a *correct* result). Without stage 3 it only *looks* like thinking.

Planning → the pause: stop and ask

The agent reasoned out the cause. But **the fix is a risky human decision** — a rollback hits users mid-checkout. The model does **not** act alone.

	Traditional software	Agent
Where it pauses	programmer hard-codes a form at that line	model decides at run-time
Who picks the question	decided in advance	model generates it on the spot

It **pauses the loop** and emits a tool call: `AskUserQuestion` .

The pause, step by step

1. Model thinking: "cause clear, but rollback is risky – human must choose."
▼
2. MODEL emits tool_use: AskUserQuestion
{ question: "How should I handle checkout-api?",
options: ["Roll back to v2.2", "Keep v2.3, raise the DB pool"] }
▼
3. CLIENT runs nothing – it RENDERS a chooser for the SRE
▼
4. SRE picks "Roll back to v2.2"
▼
5. CLIENT returns that choice to the MODEL (as a tool_result)
▼
6. MODEL continues with the decision → calls rollback_deployment

The loop waits at step 3. That wait *is* the pause.

Asking the user is just a tool call

Two tools declared **identically** — only name + args differ:

```
tools = [  
    { "name": "rollback_deployment",          # CODE tool: real effect  
      "parameters": {"properties": {"service": {...}, "to_version": {...}}}},  
    { "name": "ask_user",                     # ASK tool: human executes  
      "parameters": {"properties": {"question": {...}, "options": {...}}}},  
]
```

The branch lives in **your code**, on the tool name:

```
if call.name == "ask_user":  
    result = render_ui_and_wait(args) # ← SHOW UI, human answers (PAUSE)  
elif call.name == "rollback_deployment":  
    result = rollback_deployment(**args) # ← REAL call: hit infra API  
# send result back to the model — identical for both
```

Planning — remember

- Traditional software **executes a pre-written procedure**; an agent **reasons it out of an intent** at run-time → **reasoning** is the core skill.
- Reasoning = **draft (chain-of-thought) before answering**; more draft = more compute (**test-time compute**); learned by **RL on the result**.
- **Stop-and-ask** isn't pre-written — it **emerges** from reasoning, emitted as a tool call.
- Ask-the-user and code tools share **one mechanism**; they differ only in **who executes** + your `if` branch.

2 · Execution

Calling tools, and MCP.

The problem MCP solves

A model alone only knows what's in the conversation. It can't read your files, call an API, or touch real data.

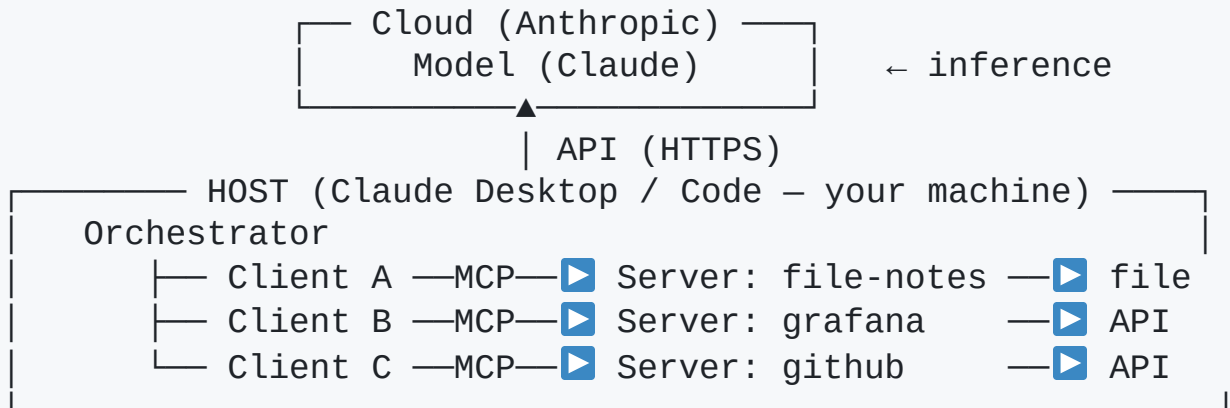
MCP (Model Context Protocol): a standard for connecting AI to external tools and data.

Write an MCP server **once** → every MCP-aware AI app can use it.

No per-model integration.

Think "USB-C for AI tools": one connector, many devices.

Four roles (the model is OUTSIDE)



- **Model** — cloud inference; never touches a server directly.
- **Host** — the app; the orchestrator bridging cloud ↔ servers.
- **Client** — one per server connection (inside the Host).
- **Server** — exposes tools/resources/prompts → real data.

The flow — creating a note

1. User: "create a note 'Project meeting', content: lock the deadline"
▼
2. MODEL picks tool `create_note {title:"Project meeting", content:"lock the deadline"}`
▼
3. CLIENT → SERVER: `tools/call create_note {...}`
▼
4. SERVER writes the .md, returns "Created note: project-meeting.md"
▼
5. CLIENT hands the result back to the MODEL
▼
6. MODEL replies: "Created the note 'Project meeting' 👍"

model wants → client calls → server does → result back → model replies.

The model emits intent; the client translates

```
// ① Model emits (its own format)
{ "type": "tool_use", "name": "create_note",
  "input": { "title": "Project meeting", "content": "lock the deadline" } }

// ② Client translates to an MCP message to the server
{ "jsonrpc": "2.0", "id": 2, "method": "tools/call",
  "params": { "name": "create_note", "arguments": { ... } } }

// ③ Server returns
{ "jsonrpc": "2.0", "id": 2,
  "result": { "content": [ { "type": "text", "text": "Created: project-meeting.md" } ] } }
```

The model **never writes JSON-RPC**. The client always normalizes to MCP → the **server sees one format** → one server works for many models.

How does the model "read" a tool?

On connect, the client asks `tools/list`; the server's descriptions are injected into the model's context. You **don't hand-write** them — FastMCP generates from the docstring + type hints:

```
@mcp.tool()
def create_note(title: str, content: str) -> str:
    """Create a new Markdown note.""" # docstring → description
```

→

```
{ "name": "create_note", "description": "Create a new Markdown note.",
  "inputSchema": { "type": "object",
    "properties": { "title": { "type": "string" }, "content": { "type": "string" } },
    "required": [ "title", "content" ] } }
```

Clear docstrings + type hints = the model's only guide for when/how to use a tool.

Tool or Resource?

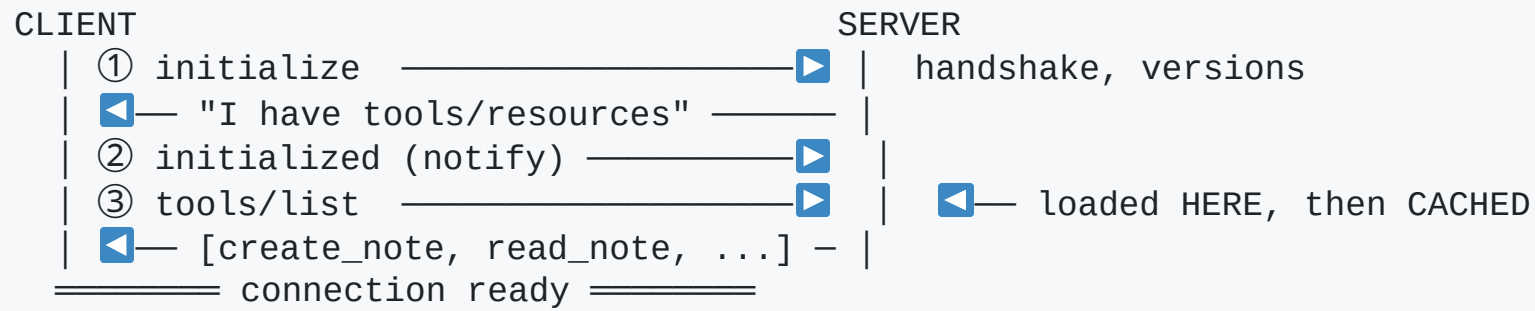
What you need	Use	Server returns
Create a note (write to disk)	tool <code>create_note</code>	"Created..."
Read an existing note for Claude	resource <code>note://...</code>	the note's contents

tool = do work · resource = read data

(Servers also offer **prompts** — pre-written command templates, e.g. `summarize_notes .`)

When is the tool list loaded? (init)

Once, at connection open — not per message.



Each chat turn reuses the **cached** list. Reload only on reconnect or `notifications/tools/list_changed`.

Local or Remote? — two independent axes

Kind	Where the server runs	Client connects via
stdio (local)	your machine, client spawns it	stdin/stdout pipe
HTTP / SSE (remote)	a far server	a URL (https://...)

- 1. *Where does the server **run**?* local vs remote
 - 2. *Where does the server **get data**?* local file vs remote API
- `file-notes` = local server + local files. Grafana MCP = local server calling a **remote API** — or a remote server hosted by Grafana.

The working example — file-notes

A Python MCP server: create/read/update/delete/search Markdown notes + file ops inside a **sandbox**.

- **Tools** — `create_note`, `read_note`, `list_notes`, `update_note`, `delete_note`, `search_notes`, `list_files`, `read_file`, `write_file`, `file_info`
- **Resources** — `note://{name}`, `notes://index`
- **Prompts** — `summarize_notes`
-  every path through `_safe_path()` → can't escape `data/`

```
mcp dev server.py      # MCP Inspector – click through each tool
claude mcp add file-notes -- /path/.venv/bin/python /path/server.py
```

Execution — remember

- MCP = write a tool **once**, any MCP-aware model uses it.
- Four roles: **Model** (cloud) · **Host** (orchestrator) · **Client** (one per server) · **Server** (tools/resources/prompts).
- Model emits *intent* → **client translates** to JSON-RPC → server sees one format.
- **tool = do** · **resource = read** · tool list loaded **once** + cached.
- "Where it runs" $\perp$ "where the data is."

3 · Review

The agent grades, verifies, and closes the loop.

Fluent ≠ correct

LLMs are **confidently fluent**, even when wrong. First drafts are usually *okay*, rarely *optimal*: missing steps, invented numbers, skipped edge cases — or **thinking it fixed the incident when it's still burning**.

NO review

"Rolled back, done!"
(believes itself)



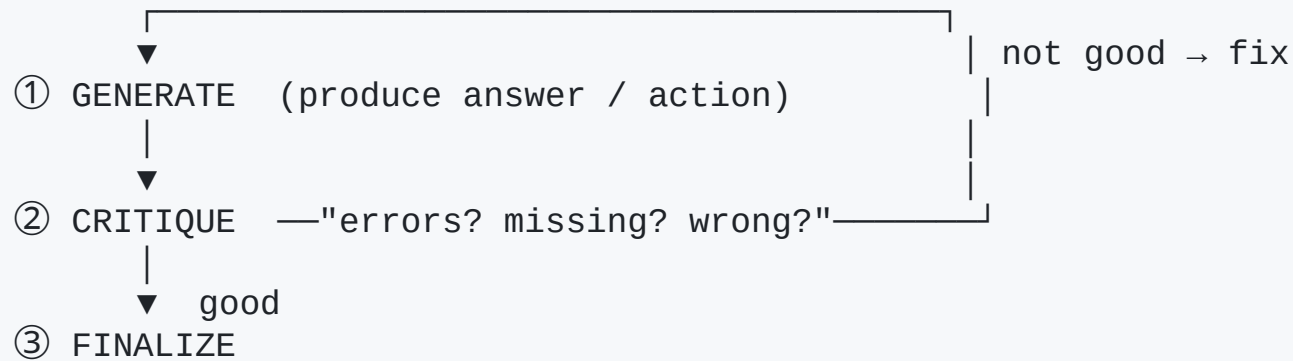
WRONG report,
incident still on fire

WITH review

"Are the 5xx really gone?"
→ verify against REAL metrics
→ still failing? → re-plan
→ clear? → only then say "fixed"

Doubt the output by default. Review turns *"seems done"* → *"verified done."*

The Reflection pattern — generate → critique → fix



Critiquing is easier than generating. "Write a perfect report" is hard; "here's a draft, find 3 weak spots" is easy. Splitting the roles gives two angles on one problem.

Two kinds of review

	Self-critique	Grounded verification
Leans on	the model's own reasoning	real-world evidence (tool/test/metric)
Question	"Is this reasoning sound?"	"Did the action actually work?"
Weakness	can be confidently wrong	needs a tool/data to check
Example	re-read, fix wording	re-query 5xx to see if it normalized

Self-critique alone is **not enough**: misunderstand the problem → critique by the same misunderstanding. The strongest review is **anchored to external truth**.

Grounded verification — the SRE agent

```
① already called rollback_deployment(checkout-api → v2.2)    [execute]
  ▼
② REVIEW: call the measuring tool AGAIN (grounded)
  query_metrics(service="checkout-api", metric="5xx_rate", window="5m")
  └─▶ 5xx = 0.1% (normal < 0.5%)
      └─✓ PASS → "Resolved. Cause v2.3 shrank DB pool, rolled back to
                v2.2, 5xx 0.1% at 14:41." → record lesson (episodic)
  └─▶ 5xx = 6% (STILL high)
      └─✗ NOT PASS → back to PLAN: "hypothesis wrong, or 2nd cause.
                Re-reason from new data."
```

The agent may not take credit for itself — re-measure the *same signal* that raised the alarm.

Who reviews? — itself, a judge, or hard checks

1. **Self-review** — model re-reads its own output. Cheap; blind to its own systematic errors.
2. **LLM-as-judge** — a *second* model grades against a rubric. Independent angle; the common way to auto-grade at scale.
3. **Deterministic checks** — no model: tests, schema validation, lint, metric thresholds, policy gates. Most trustworthy.

Whatever a machine can check, don't ask the model to "feel."

Judge model → qualitative (text, argument). Hard checks → quantitative (tests, $5xx < \text{threshold}$, valid JSON).

Closing the loop — and the brakes

```
review —┬─ PASS —▶ finalize → write memory → DONE
          └─ NOT PASS —▶ feed errors into planning → re-think → review again
```

The loop **must have a brake**, or it fixes-grades-fixes forever (or repeats a bad action on prod). Stop on:

- **Criteria met** — success (5xx under threshold, tests green)
- **Budget exhausted** — too many rounds / tokens / time
- **No progress** — new round not measurably better
- **Beyond authority** — **escalate to a human** (another `AskUserQuestion`)

Review — remember

- Output is **fluent** $\neq$ **correct** → doubt it before finalizing.
- **Reflection** = generate → critique → fix; *critiquing is easier than generating*.
- **Self-critique** (own reasoning, can be confidently wrong) vs **grounded verification** (anchored to truth). Re-measure 5xx, don't self-certify.
- Reviewer: **self / judge model / hard checks** — machine-checkable → don't "feel" it.
- Review **closes the loop** and **must have stopping conditions**.

4 • Memory

The four kinds every agent needs.

Why memory? — an LLM is stateless

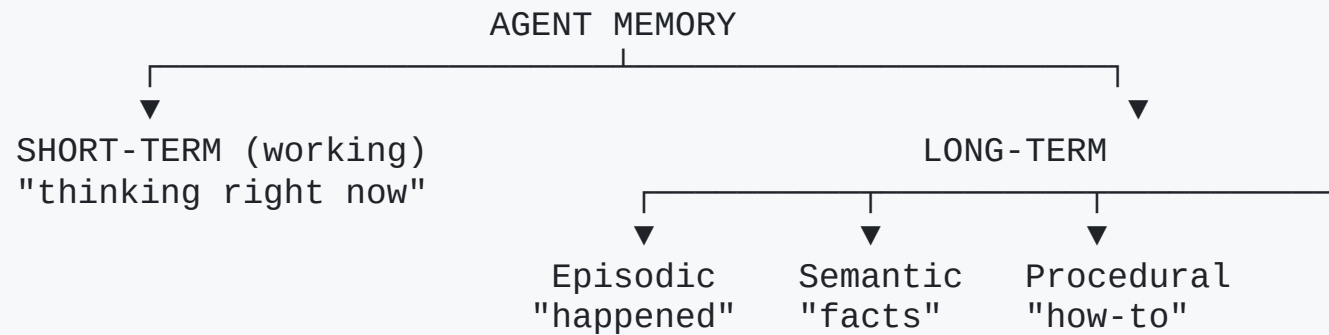
Each call sees only what's in the context, then **forgets everything**. End of session → forgets your name, what it just did, the lesson from a turn ago.

An **agent** acts across many steps, turns, sessions. It needs memory to:

- hold the current context
- recall what happened
- know facts (world + you)
- know *how* to do things

Borrowing from human cognitive science: **four kinds**, and an agent needs all four.

The four types



working = RAM now · episodic = diary · semantic = knowledge · procedural = skills

1 · Working · 2 · Episodic

Working memory (short-term) — the *current task's* context: recent messages, last tool result, current goal. It **is** the context window (RAM).

- Limit: token count → full means trim/summarize. Lost at session end.

Episodic memory (long-term) — **specific events**: what, when, with what, outcome. *A diary.*

- Used for **case-based reasoning** — learn from a prior case; few-shot from history.
- Stored in indexed logs / old sessions.
- "*Last week you preferred a short table*" → does it again.

3 · Semantic · 4 · Procedural

Semantic memory (long-term) — **generalized facts**, not tied to an event:

"Paris is the capital of France," "customer A is Enterprise."

- Used for **grounding** — anchor answers in fact, not invention.
- Stored in vector DB (RAG), knowledge graph, user profile.

Procedural memory (long-term) — **how-to**: steps, tool conventions, the "rules of the game." Often *not* in data but in **system prompt, code, weights**.

- Used to execute multi-step tasks consistently.

Episodic = *what happened* · Semantic = *what's true* · Procedural = *how to do it*.

Quick comparison

Type	Span	Answers	Stored in
Working	Short	"What am I doing now?"	Context window
Episodic	Long	"What happened before?"	Indexed logs / old sessions
Semantic	Long	"What is true?"	Vector DB (RAG), knowledge graph
Procedural	Long	"How do I do this?"	System prompt, code, tools, weights

Write and retrieve — both directions

EXPERIENCE → [write] → long-term store → [retrieve at the right moment]
→ inject into working memory → model uses it

- **Write:** after each turn, distill what's worth keeping (don't store everything — the *non-trivial*).
- **Retrieve:** find the **relevant** memory (similarity search) and load it back into working memory.

Working memory is finite → **choosing what to load** is the crux.

Type vs Scope — two independent axes

- **Type** (working / episodic / semantic / procedural) = the *content*.
- **Scope** = *who/where it applies*: session → user → agent → org .

Any type sits at any scope. Episodic, for example:

Scope	Episodic at that scope
Session	events within this session (gone after)
User	a diary across all sessions, all projects
Agent / Org	events shared across agents / a team

Episodic isn't bound to one project — a real system keeps it **user-level, cross-session**.

Local demo (Claude Code) → production (mem0)

Claude Code alone (files on disk) demos **all four** — single, local, one person:

Type	Claude Code uses
Working	session context window (auto-summarizes)
Semantic	fact files in <code>memory/</code> + <code>MEMORY.md</code>
Episodic	old transcripts (<code>--resume</code>) + <code>feedback</code> / <code>project</code> files
Procedural	<code>CLAUDE.md</code> + skills + system prompt + tools

At **multi-agent production**, hand-managed files break in 3 ways → **mem0**:

- 1. **Scope & sharing** — keyed by user/session/agent/org; many agents share one store.
- 2. **Smart retrieval at scale** — vector + graph; `grep` can't keep up at thousands.
- 3. **Auto write/merge** — distill, dedupe, reconcile contradictions automatically.

Memory — remember

- An LLM is **stateless**; an agent needs memory to persist across steps/sessions.
- Four types: **working** (RAM now) · **episodic** (diary) · **semantic** (facts) · **procedural** (how-to).
- **Type** and **scope** (session/user/agent/org) are **independent** axes.
- Long-term memory needs **write** (distill) + **retrieve** (load the relevant bit).
- Files demo all four; **mem0** is the production layer for scope, retrieval, and auto-merge.

5 · Putting it together

The loop, end to end.

One incident, the whole loop

14:35 alert: checkout-api 5xx spike. SRE: "deal with it."

PLAN reason: 5xx since 14:32 → deploy v2.3 at 14:30 → logs: DB
timeout → v2.3 shrank the pool. Cause = the deploy.
Rollback is risky → AskUserQuestion → "roll back to v2.2"



EXECUTE call rollback_deployment(checkout-api → v2.2) [MCP]



REVIEW grounded: query_metrics(5xx, 5m) → 0.1% 
(still 6%? → back to PLAN with new data)



MEMORY episodic: "v2.3 shrank DB pool → 5xx; rollback fixed it"
→ next time, this class of incident resolves faster

Takeaways

1. A model is a **chat box**; the **loop** (plan → execute → review → memory) makes it an **agent**.
2. **Planning** = reasoning: draft before answering, spend test-time compute, stop-and-ask when it's a human's call.
3. **Execution** = tool calling via **MCP**: write once, use everywhere.
4. **Review** = verify against **truth**, not the model's confidence; always have a **brake**.
5. **Memory** = four types × scope; **write + retrieve**; files → mem0.
6. **Use both kinds** — graft an agent onto your existing deterministic workflow: it reads your alerts and **reduces toil**; the static layer guards, the agent reasons.

Thank you

Questions?

Full notes & sources: this folder's `README.md`